

Deep Learning Midterm Project Report

Che Hung Liao
cl8172@nyu.edu

Eleftherios Komvopoulos
ek4538@nyu.edu

Abstract

This report presents our approach to a deep learning task using modern neural network methods. We focus on building a competitive model under practical training-time constraints, compare multiple configurations, and summarize the design and optimization choices that contributed most to performance.

1 Introduction

This project focuses on solving a deep learning task using modern neural network approaches. The goal is to build a model that achieves strong performance on the given dataset while exploring different architectural and training choices.

Due to the computational cost of training, we emphasize iterative experimentation and practical improvements over exhaustive search. We evaluate different configurations and analyze what contributes most to performance.

1
2
3

2 Dataset

[TO BE FILLED]

We describe the dataset, preprocessing steps, and any augmentation techniques applied.

3 Model Description

We build on the Qwen2.5 series as our base model family, which provides strong general-purpose capabilities and a range of parameter sizes suitable for experimentation on limited hardware. For this task we chose the Qwen/Qwen2.5-Coder-XB-Instruct variants because they are optimized for

code-oriented reasoning and instruction-following behavior, aligning well with the debugging, code-related structured output, and prompt-driven workflow required by the competition setting. We initially evaluated 1.5B, 3B, and 7B parameter models; however, the 7B version consistently pushed GPU memory to near out-of-memory limits, so we ultimately standardized on 3B and 1.5B models as the best balance between capacity and stable training/inference.

4 Experimentation

Our experimentation followed a staged and hypothesis-driven progression. We began with the official starter implementation and first modified it so that it could run reliably on a single AMD MI100 GPU, since the original workflow was not directly aligned with our hardware environment. Using this adapted starter as the baseline, we conducted a sequence of controlled ablations on the Qwen2.5-1.5B model family. We first increased the maximum sequence length from 1536 to 2048 to test whether longer context windows would improve SVG generation quality. We then increased the LoRA rank and LoRA scaling factor to evaluate whether stronger low-rank adaptation would improve performance. Finally, we expanded the training set size in stages, moving from the default subset to 24,000 and then 36,000 samples, in order to determine whether additional supervision translated into better results. These initial experiments established a clear reference point, but they did not yield a meaningful improvement over the starter baseline.

We therefore shifted to Kaggle's NVIDIA T4 $\times$ 2 environment and moved to the Unsloth 3B model family. In this phase, we used 4-bit loading so that the larger 3B models would remain trainable within the memory limits of the hosted GPUs while reducing the risk of out-of-memory failures. Start-

¹Code repository: github.com/shanpig/dl-kaggle-midterm.

²Model weights: [ADD YOUR LINK HERE]

³We used GPT and Cursor for debugging and implementation assistance.

ing from an initial 3B configuration, we increased LoRA rank and per-device batch size to better utilize the available GPU resources and improve training throughput. We then introduced a more explicit system prompt containing SVG-specific rules to test whether stronger task guidance improved both training behavior and inference quality. In a later Kaggle variant, we further increased the per-device batch size, doubled the learning rate, and doubled the number of training epochs in order to test a more aggressive optimization setup while remaining within the platform’s memory constraints.

As the project progressed, Kaggle usage limits became a practical bottleneck for longer and more resource-intensive runs. We therefore returned to the AMD platform, this time using a two-GPU setup for our final high-capacity experiment. In this last stage, we increased the maximum sequence length to 3072 and switched to Qwen2.5-Coder-3B-Instruct, while also lowering the LoRA rank, increasing the LoRA scaling factor, and training for more epochs. This configuration was intended to combine longer context, stronger code-oriented reasoning, and a more stable training environment for extended runs. Among the configurations we evaluated, this final AMD-based setup produced the strongest overall result.

5 Results

The final model achieved improved performance compared to the baseline. We observed that tuning hyperparameters, particularly learning rate and model configuration, had a noticeable impact on results.

In some cases, increasing model complexity did not yield proportional improvements, indicating diminishing returns.

6 Conclusion

In this project, we explored different modeling approaches and training strategies to improve performance on the task.

We found that careful tuning of hyperparameters and iterative experimentation were key to achieving better results. However, training time posed a significant constraint, limiting the number of experiments that could be conducted.

Some approaches did not lead to improvements, highlighting the importance of targeted experimentation rather than blindly increasing complexity.

In future work, we would implement better experiment tracking and explore more efficient training methods to allow broader exploration of the design space.

Experiment	Model	Max Seq	r	α	LR	Epochs	Train BS	Eval BS	Grad Accum	Target Samples	Platform
starter/1536	Qwen2.5-1.5B-Instruct	1536	16	16	2e-4	1	1	1	8	12000	AMD MI100
starter/2048	Qwen2.5-1.5B-Instruct	2048	16	16	2e-4	1	1	1	8	12000	AMD MI100
starter/2048-r32	Qwen2.5-1.5B-Instruct	2048	32	64	2e-4	1	1	1	8	12000	AMD MI100
starter/2048-r32-24000	Qwen2.5-1.5B-Instruct	2048	32	64	2e-4	1	1	1	8	24000	AMD MI100
starter/2048-r32-36000	Qwen2.5-1.5B-Instruct	2048	32	64	2e-4	1	1	1	8	36000	AMD MI100
2gpu version	Qwen2.5-3B-Instruct-bnb-4bit	2048	16	16	2e-4	1	1	—	4	—	Kaggle NVIDIA T4 $\times$ 2
2gpu-version-batch2	Qwen2.5-3B-Instruct-bnb-4bit	2048	32	16	2e-4	1	2	—	4	—	Kaggle NVIDIA T4 $\times$ 2
2gpu-version-batch2-with-prompt	Qwen2.5-3B-Instruct-bnb-4bit	2048	32	16	2e-4	1	2	—	4	—	Kaggle NVIDIA T4 $\times$ 2
2gpu-version-batch32-2xlr-step4	Qwen2.5-3B-Instruct-bnb-4bit	2048	32	16	4e-4	2	4	—	4	—	Kaggle NVIDIA T4 $\times$ 2
3072-17hrs	Qwen2.5-Coder-3B-Instruct	3072	16	64	2e-4	3	4	1	4	20000	AMD MI100 $\times$ 2

Table 1: Experiment configurations (scores omitted here).

Model	Score
Baseline	[XX.X]
Final Model	[XX.X]

Table 2: Comparison between the baseline and final model performance.